

UNIQUE: Universal Top-k Sparse Attention for Training-free Inference and Sparsity-aware Training

Keqi Deng, Shaoshi Ling, Ruchao Fan, Jinyu Li

Microsoft, USA

keqideng@microsoft.com

Abstract

Long-context inference in large language models (LLMs) is bottlenecked by the linear growth of the self-attention key-value (KV) cache. Top- k sparse attention alleviates this by loading only a small fraction of the KV cache, but accurately and cheaply estimating cache importance, for both training-free use and sparsity-aware training, remains challenging. This paper proposes UNIQUE, a universal top- k sparse attention framework that addresses both requirements and stays consistently effective across LLM modalities. UNIQUE operates at the granularity of KV pages and estimates per-page importance with a simple yet accurate score combining the mean of the page’s keys as a representative vector with their standard deviation as an offset term. To further close the train-inference gap, this paper introduces a soft-mask sparsity-aware training scheme that uses the top- k score boundary as a per-query threshold and a sigmoid soft mask around it, requiring neither auxiliary losses nor architectural changes. Experiments on text and speech LLMs show that UNIQUE preserves task performance on long-context benchmarks such as LongBench Pro and on long-form speech recognition, while delivering up to $11.4\times$ attention-kernel speedup over FlashInfer dense attention and at least $5.3\times$ end-to-end decoding speedup over a vLLM-based dense model.

1 Introduction

Large language models (LLMs) have rapidly expanded their context windows from a few thousand to hundreds of thousands of tokens (Achiam et al., 2023; Dubey et al., 2024; Comanici et al., 2025; Abouelenin et al., 2025), unlocking applications such as long-document understanding, repository-level code reasoning, and long-horizon multi-turn agents. This growing context length, however, makes the key value (KV) cache rather than model weights the dominant source of memory traffic

during auto-regressive decoding: each generated token requires reading the entire KV cache from high-bandwidth memory, and the size of this cache grows linearly with sequence length (Pope et al., 2023; Deng and Woodland, 2025). At hundred-thousand-token scales, the per-token KV cache read can dominate decoding latency and cap end-to-end throughput, regardless of how fast the matrix multiplications themselves are (Dao, 2024; Kwon et al., 2023; Ye et al., 2025). The same pressure is no longer confined to text. Speech and audio LLMs increasingly operate on hours-long recordings for meeting transcription, podcast understanding, and streaming dialogue (Park et al., 2025; Peng et al., 2025), where a single recording can already produce a speech sequence of tens of thousands of positions and stress the KV cache in the same way as long-context text. Reducing the cost of long-context attention is therefore a shared bottleneck across modalities rather than a text-only concern.

To address the KV cache bottleneck, sparse attention drops less informative tokens to compute attention over a small context subset. Existing approaches typically follow one of two paradigms. Training-free methods (Zhang et al., 2023; Xiao et al., 2024; Tang et al., 2024) dynamically select top- k KV pairs without updating model weights. While they offer easy deployment, a mismatch between dense training and sparse inference often leads to notable performance degradation under high sparsity. Alternatively, trainable sparse mechanisms such as Native Sparse Attention (NSA) (Yuan et al., 2025b) and its variants (Zhao et al., 2025) mitigate this drop by incorporating sparsity directly into model training. However, these methods typically rely on specialized architectural components or auxiliary routing modules, limiting their flexibility and often demanding resource-intensive training. In practice, a unified framework that provides immediate training-free acceleration while remaining optimizable through

sparsity-aware fine-tuning is more desirable. Moreover, current sparse attention strategies are almost exclusively validated on text LLMs, leaving their robustness on continuous modalities such as long-form speech in speech LLMs largely unexplored.

This paper proposes UNIQUE, a universal top- k sparse attention framework that supports both training-free deployment and sparsity-aware training across modalities. For hardware efficiency, UNIQUE operates at KV-page granularity (Kwon et al., 2023). For each page, UNIQUE estimates its importance with a simple yet accurate score. It first computes the mean of all keys within the page as a representative vector. Considering that the mean alone may dilute the salience of a few highly informative keys, UNIQUE introduces an additional offset term based on the standard deviation of the keys, which compensates for the underestimated importance. This offset-augmented score surpasses existing training-free sparse attention methods across modalities. To further close the gap between dense training and sparse inference, the top- k decision boundary serves as a per-query threshold, and a sigmoid function is applied to derive a differentiable soft mask. This allows the model to adapt to sparse attention and actively refine its top- k selection during training, requiring no auxiliary loss or architectural change. Experiments on long-context text benchmarks and long-form speech recognition show that UNIQUE preserves task quality while significantly accelerating decoding.

The main contributions of this paper can be summarized in four parts:

- A universal top- k sparse attention framework, UNIQUE, that unifies training-free and trainable sparse attention in a single design, validated on both text and speech modalities.
- An offset-augmented page scoring method that combines the mean and standard deviation of keys within each page, achieving more accurate importance estimation than existing approaches.
- A sigmoid soft-mask scheme that derives a differentiable gate from the top- k decision boundary, enabling sparsity-aware training without auxiliary losses or extra parameters.
- Efficient CUDA implementations including fused kernels and optimized operators, e.g., a top- k selection that is at least $2\times$ faster than PyTorch and FlashInfer counterparts.

2 Related Work

2.1 KV Cache Reduction

Reducing the memory footprint of the KV cache has been extensively studied at the architecture level. MQA (Shazeer, 2019) and GQA (Ainslie et al., 2023) reduce the number of key-value heads, with GQA retaining a moderate number of groups to trade off between cache size and representational capacity. MLA (Liu et al., 2024) compresses all key-value heads into a single low-rank latent, effectively recovering MQA-level cache cost while surpassing GQA in expressiveness.

Orthogonal to architectural changes, token compression methods reduce KV cache size by permanently discarding or condensing tokens deemed less important. H2O (Zhang et al., 2023) evicts KV entries with low accumulated attention scores, and SnapKV (Li et al., 2024) prunes tokens based on observed attention patterns. Rather than outright removal, beacon-based approaches (Zhang et al., 2025b) insert summary tokens that compress preceding chunks into compact representations. However, as analyzed in Quest (Tang et al., 2024), such compression is inherently irreversible: once a token is discarded or merged, the original information becomes inaccessible to subsequent queries. In contrast, sparse attention retains the full KV cache and dynamically selects a query-specific subset, preserving access to any past token.

2.2 Sparse Attention

Training-free methods Training-free sparse attention exploits the inherent sparsity of attention in dense-trained models. Quest (Tang et al., 2024) organizes keys into pages and selects top- k pages per query via a min-max method, while InfLLM (Xiao et al., 2024) extends this to million-length contexts by offloading distant pages to CPU. MInference (Jiang et al., 2024) and FlexPrefill (Lai et al., 2025) target the prefilling phase; this paper instead focuses on decoding, which dominates end-to-end latency as reported by Tang et al. (2024) (e.g., over 86% for 16k-token prompts with 512-token responses). SpargeAttention (Zhang et al., 2025a) and BLASST (Yuan et al., 2025a) skip value loading based on key importance; BLASST is capped at $2\times$ since keys are still fully loaded, and SpargeAttention requires per-task offline calibration of its sparse setup. Overall, training-free methods often stay at conservative sparsity to avoid degradation, limiting the achievable acceleration.

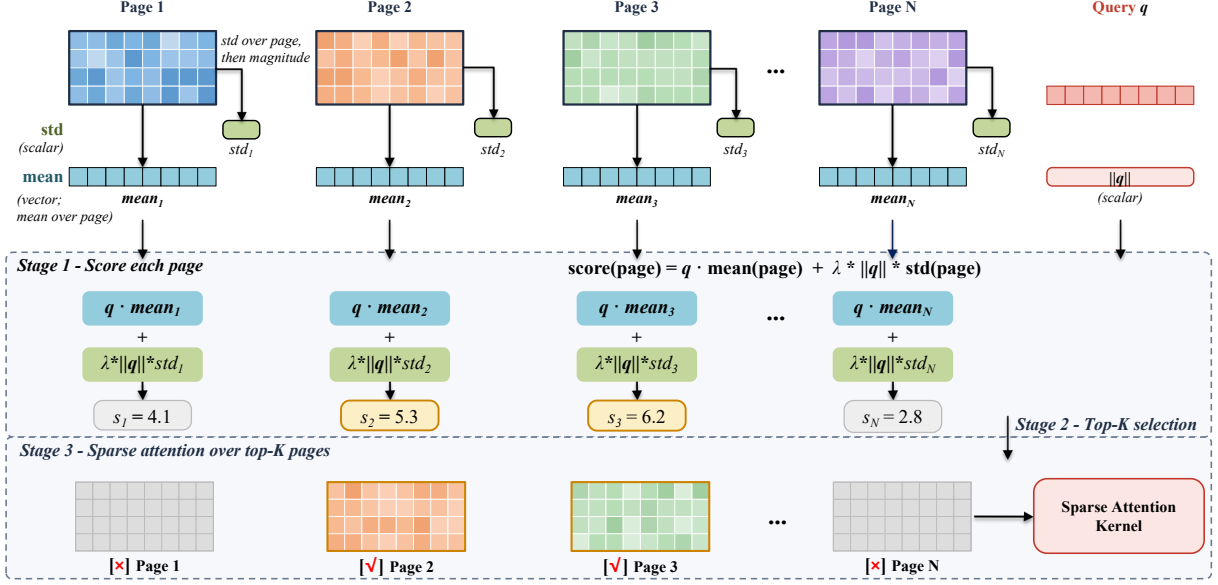


Figure 1: Overview of page criticality estimation in UNIQUE. For each KV page, UNIQUE precomputes a key mean (a vector) and a scalar standard deviation (std) of its keys. Stage 1 combines them with the query into a per-page score, Stage 2 selects the top- k pages, and Stage 3 sends only these pages to the sparse attention kernel.

Trainable methods Trainable sparse attention incorporates sparsity directly into training, allowing models to actively adapt to sparse patterns. NSA (Yuan et al., 2025b) introduces a tri-branch design with compressed, selected, and sliding attention, achieving strong performance but requiring training from scratch. InfLLM-v2 (Zhao et al., 2025) simplifies NSA by removing the compressed branch, enabling fine-tuning from dense checkpoints. SeerAttention (Gao et al., 2025) instead trains a lightweight router via self-distillation to predict block-level sparsity. In contrast, UNIQUE proposed in this paper requires no auxiliary loss or extra architectural components, while supporting both training-free deployment and sparsity-aware fine-tuning in a single unified design.

3 Methodology

This section presents UNIQUE, a page-level top- k sparse attention framework that performs sparse decoding without any training, and can be further refined when sparsity-aware training data is available. The page criticality estimator at the core of UNIQUE is built from two cheap statistics of each page’s keys (Section 3.1), supported by a fused pipeline that turns the abstraction into real wall-clock speedup at inference (Section 3.2). When training is allowed, a differentiable sigmoid mask is applied around the top- k boundary, letting the model adapt to sparsity without any auxiliary loss or extra parameters (Section 3.3).

3.1 Page Criticality Estimation

UNIQUE partitions the KV cache into contiguous pages following Kwon et al. (2023) and assigns each page a single criticality score. Let the cache contain N keys at the current decoding step, partitioned into $P = \lceil N/S \rceil$ contiguous pages of size S . The p -th page is denoted as $\mathbf{K}_p = \{\mathbf{k}_{p,1}, \dots, \mathbf{k}_{p,S}\}$ with $\mathbf{k}_{p,j} \in \mathbb{R}^D$, and the current decoding query is denoted as $\mathbf{q} \in \mathbb{R}^D$. The goal of UNIQUE is to assign each page p a single scalar score $\text{score}(p)$, so that only the top- k scoring pages are attended to during the actual attention computation. The full pipeline is illustrated in Fig. 1.

For every page, UNIQUE precomputes two cheap statistics that depend only on its keys and can therefore be cached once and reused across all subsequent decoding steps. The first is the per-dimension key mean of the page,

$$\text{mean}_p = \frac{1}{S} \sum_{j=1}^S \mathbf{k}_{p,j} \in \mathbb{R}^D, \quad (1)$$

which serves as a single representative vector of the keys inside the page. The second statistic measures how dispersed the keys are around this mean,

$$\text{std}_p = \left\| \sqrt{\frac{1}{S} \sum_{j=1}^S (\mathbf{k}_{p,j} - \text{mean}_p)^2} \right\|_2 \in \mathbb{R}, \quad (2)$$

where the per-dimension standard deviation is first computed, and then collapsed into a single scalar per page through its ℓ_2 norm.

Given the query $\mathbf{q}$, each page is then scored by combining the two precomputed statistics with a single hyperparameter $\lambda \geq 0$:

$$\text{score}(p) = \mathbf{q} \cdot \mathbf{mean}_p + \lambda \|\mathbf{q}\|_2 \text{std}_p, \quad (3)$$

with λ fixed to 0.5 in all experiments. The first term $\mathbf{q} \cdot \mathbf{mean}_p$ measures how well the query aligns with the average key of the page, but the mean alone is prone to dilution: when a page contains a few highly informative keys mixed with many uninformative ones, averaging washes the informative ones out and the page receives only a moderate score. The second term $\lambda \|\mathbf{q}\|_2 \text{std}_p$ counteracts this by lifting the scores of pages whose keys are spread out, since such pages are more likely to hide a sharp, query-relevant key. The multiplication by $\|\mathbf{q}\|_2$ keeps the two terms on a comparable scale across layers and queries, without introducing any learnable parameter.

3.2 Inference and Efficient Implementation

At decoding time, let $\mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times D}$ denote the keys and values currently stored in the KV cache and $\mathbf{q} \in \mathbb{R}^D$ denote the decoding query. The dense attention output is

$$\mathbf{o} = \text{softmax}\left(\frac{\mathbf{q}\mathbf{K}^\top}{\sqrt{D}}\right) \mathbf{V}. \quad (4)$$

UNIQUE replaces this with a sparse counterpart that only reads the keys and values of the top- k pages chosen by Eq. 3. Let $\mathcal{T}_k(\mathbf{q})$ be the indices of those pages, and let $\mathbf{K}_{\mathcal{T}_k(\mathbf{q})}, \mathbf{V}_{\mathcal{T}_k(\mathbf{q})} \in \mathbb{R}^{k \times D}$ stack their keys and values along the sequence axis. The sparse attention output is then

$$\mathbf{o} = \text{softmax}\left(\frac{\mathbf{q}\mathbf{K}_{\mathcal{T}_k(\mathbf{q})}^\top}{\sqrt{D}}\right) \mathbf{V}_{\mathcal{T}_k(\mathbf{q})}. \quad (5)$$

Each decoding step of UNIQUE thus decomposes into three CUDA operations: criticality estimation, top- k selection, and sparse attention.

Fused criticality estimation kernel Since GQA is adopted in this paper, pages are partitioned along the KV-head dimension, and the top- k selection is performed independently for each of the KV heads H_{kv} . Within each group, the $G = H/H_{kv}$ query heads $\{\mathbf{q}_g\}_{g \in \mathcal{G}}$ sharing the same KV head produce G raw scores against every page, and UNIQUE aggregates them by taking the maximum across the group:

$$\text{score}(p) = \max_{g \in \mathcal{G}} [\mathbf{q}_g \cdot \mathbf{mean}_p + \lambda \|\mathbf{q}_g\|_2 \text{std}_p], \quad (6)$$

so a page is kept if any query head in the group finds it relevant, consistent with GQA semantics.

A naive realization of Eq. 6 performs three separate launches: a batched matmul for the $G \times P$ raw scores per KV head, an elementwise add for the per-head offset $\lambda \|\mathbf{q}_g\|_2 \text{std}_p = \lambda (\|\mathbf{q}\|_{1:G}) (\text{std}_{1:P})^\top$, which is a rank-one outer product, and a max reduction across the G query heads. Each launch round-trips the $G \times P$ intermediate score tensor through high-bandwidth memory (HBM), which dominates the cost when P is large. UNIQUE fuses all three steps into a single CUDA kernel: a batched matmul produces the $G \times P$ raw scores, a fused epilogue applies the rank-one offset and the max reduction without ever materializing the full $G \times P$ offset tensor in HBM, and only the final P -vector of per-page scores is written back. This roughly halves the kernel’s memory traffic. On an H100 GPU, this fusion alone yields up to **1.73** $\times$ speedup over the naive unfused implementation. More details refer to Appendix A.

Radix-based top- k selection Once the per-page scores are available, UNIQUE extracts the k largest of them for each KV head. Generic top- k routines in deep-learning frameworks leave room for improvement here when $k \ll P$ and the subsequent sparse attention does not need the top- k pages in sorted order. UNIQUE therefore reformulates the problem as a two-round 8-bit radix selection over bf16 scores: each round builds a 256-bin histogram in shared memory, locates the bucket containing the k -th largest element, and shrinks the search range by one byte. With only two rounds the total work is $O(P)$ instead of $O(P \log P)$, and the entire computation for one row fits in a single CUDA block. The implementation is specialized for three regimes: for $P \leq 4096$ all scores are cached in registers and shared memory is reserved for the histograms; for larger P the scores are staged in shared memory, which accommodates up to $P \approx 23\text{K}$ pages under the default 48 KB per-block budget shared by all recent NVIDIA GPUs; beyond that, UNIQUE falls back to the top- k routine of FlashInfer (Ye et al., 2025). Selected physical page IDs are gathered via the page table inside the same kernel, so no intermediate index tensor reaches HBM. On an H100 GPU this implementation runs up to $2.0\times$ faster than `flashinfer.top_k` and $4.8\times$ faster than `torch.topk`; see Appendix B for details.

Sparse attention kernel After top- k selection, the sparse attention reduces to a standard paged

attention call: for each query head, the keys and values of the selected pages are read from the KV cache and attended to. Since UNIQUE selects whole pages of contiguous keys and values, each selected page can be loaded with coalesced memory accesses by the attention kernel, exactly as in standard paged attention (Ye et al., 2025), where the selected pages are streamed from HBM into shared memory and $\mathbf{q}\mathbf{K}^\top$, softmax, and reweighting of $\mathbf{V}$ are performed entirely on chip. The page-level granularity of UNIQUE composes naturally with this kernel: switching from dense to sparse paged attention requires only passing in a different set of page IDs, and the cost of this step scales with the token budget $k \cdot S$ rather than the KV length N .

3.3 Sparsity-aware Training

While UNIQUE already works well in the training-free setting, it further supports a sparsity-aware training scheme that not only adapts the model to sparse attention but also trains the criticality estimation itself, without any auxiliary loss or architectural change. For brevity, let $s_p \equiv \text{score}(p)$, and let $s_{(1)} \geq s_{(2)} \geq \dots \geq s_{(P)}$ denote these scores sorted in descending order. The midpoint of the k -th and $(k+1)$ -th largest scores is taken as the per-query top- k boundary,

$$\theta = \text{sg}\left(\frac{s_{(k)} + s_{(k+1)}}{2}\right), \quad (7)$$

where $\text{sg}(\cdot)$ stops the gradient. A differentiable soft mask is defined for each page as

$$g_p = \sigma\left(\frac{\text{score}(p) - \theta}{\tau}\right), \quad (8)$$

where the query norm $\|\mathbf{q}\|_2$ inside $\text{score}(p)$ is detached from gradient for training stability, and g_p is applied as an additive log-bias $\log g_p$ inside the attention softmax at training, so that the gradient flows back to the page scores through g_p .

This boundary choice concentrates training signal where it is most useful. The derivative $\partial g_p / \partial s_p = g_p(1 - g_p) / \tau$ peaks at $s_p = \theta$ and decays rapidly away from it, so pages that are clearly inside or outside the top- k receive negligible gradient while pages near the boundary are pushed apart. Sparsity-aware training therefore widens the margin around the top- k cutoff rather than perturbing already-saturated scores. At the end of training, the soft mask can be switched to a hard top- k mask for a small number of steps, which further aligns training with the exact selection used at inference.

The page-level soft mask $g_p \in \mathbb{R}^{Q \times P}$ is much smaller than the $\mathbb{R}^{Q \times N}$ mask used in token-level schemes, and is fused into a FlashAttention-style kernel that performs the gate, the softmax, and their backward passes in a single block-wise pass without materializing the full attention matrix. Sparsity-aware fine-tuning therefore runs at wall-clock cost comparable to dense fine-tuning while still propagating gradients to the page scores.

4 Experiments

4.1 Setup

UNIQUE is evaluated on two modalities, long-context text understanding and long-form automatic speech recognition (ASR). For text, training-free results are reported on LongBench-Pro (Chen et al., 2026), a bilingual benchmark of 1,500 English and Chinese samples covering 11 primary and 25 secondary tasks with inputs up to 256K tokens, and on the 32K subset of RULER (Hsieh et al., 2024), which spans retrieval, multi-hop tracing, aggregation, and question answering. Sparsity-aware fine-tuning uses LongAlign-10k (Bai et al., 2024), a primarily English long-context corpus of about 10K instances; fine-tuned models are therefore evaluated on the English subset of LongBench-Pro. For speech, 69K hours of Portuguese ASR data (utterances up to one hour) are used to both train the dense speech LLM and conduct sparsity-aware fine-tuning, and evaluation is performed on long-form Portuguese ASR test sets of 10-minute utterances.

Ministral-3-8B-Instruct-2512 (Liu et al., 2026) is used as the text LLM. The speech LLM uses Qwen3-8B (Yang et al., 2025) as the language backbone, coupled with a Conformer speech encoder.

A series of popular sparse-attention methods of both flavors are implemented for comparison with the proposed UNIQUE. The **training-free** methods include H2O (Zhang et al., 2023), Quest (Tang et al., 2024), and InfLLM (Xiao et al., 2024); the **trainable** methods include InfLLM-v2 (Zhao et al., 2025) and DeepSeek Sparse Attention (DSA) (Liu et al., 2025). All methods share the same LLM backbone and token budget as UNIQUE for a fair comparison, with the budget set to 512 unless otherwise stated. More details refer to Appendix C.

4.2 Long-Context Text Understanding

Training-free results Table 1 and Table 2 report LongBench-Pro and RULER-32K accuracy of Ministral-3-8B-Instruct-2512 under per-query

Method	Budget	Overall	En	Zh	Extreme	Hard	Moderate	Easy
Full attention	–	37.70	35.88	38.96	31.13	33.72	31.60	48.86
Quest	2048	30.63	29.17	32.60	24.60	28.60	24.17	42.01
H2O	2048	33.02	31.77	34.42	27.51	31.08	26.56	43.33
InfLLM	2048	36.16	35.06	37.53	29.95	32.90	29.84	47.98
Proposed UNIQUE	2048	37.11	35.57	38.15	31.28	33.18	31.03	47.66
Quest	1024	27.18	25.83	28.76	21.53	22.69	21.62	38.71
H2O	1024	31.61	30.59	33.16	26.93	29.15	25.43	41.87
InfLLM	1024	35.42	34.77	36.68	29.21	32.58	28.60	48.12
Proposed UNIQUE	1024	36.72	35.77	37.98	31.02	32.78	30.75	48.34
Quest	512	21.72	21.09	21.84	18.87	16.18	17.61	29.31
H2O	512	29.04	27.96	30.44	23.61	28.15	22.20	39.13
InfLLM	512	34.99	33.82	36.21	28.96	32.38	27.31	46.74
Proposed UNIQUE	512	36.58	35.22	38.23	30.86	33.75	30.37	47.67
Quest	256	15.41	15.65	14.67	14.78	11.34	12.66	19.29
H2O	256	26.47	25.76	26.96	19.05	25.42	19.58	37.67
InfLLM	256	33.51	32.87	34.08	27.59	30.39	25.62	45.40
UNIQUE	256	35.91	35.11	36.88	30.23	33.38	29.11	46.94
Quest	128	8.36	9.56	7.75	9.85	5.59	6.73	10.55
H2O	128	24.25	23.74	24.68	14.10	24.71	18.85	36.35
InfLLM	128	30.88	30.39	31.11	25.50	29.89	23.38	40.49
Proposed UNIQUE	128	34.70	33.47	36.41	28.90	31.97	29.00	45.80

Table 1: Training-free LongBench-Pro results on Ministral-3-8B-Instruct-2512 (higher is better, $\uparrow$). **Budget** denotes the number of KV tokens each query is allowed to attend to at decoding time; full attention attends to all KV tokens.

Method	Budget	SG1	SG2	SG3	MK1	MK2	MK3	MV	MQ	VT	CWE	FWE	QA1	QA2	AVG
Full attention	–	100.00	99.60	99.60	99.40	100.00	99.80	98.55	99.05	99.96	88.46	93.27	75.80	57.40	93.15
Quest	2048	100.00	98.80	97.00	99.20	99.80	97.60	99.45	99.30	99.88	77.80	90.67	75.00	57.20	91.67
H2O	2048	100.00	99.80	31.00	99.60	97.80	1.80	92.65	65.20	98.40	42.96	90.80	74.40	56.80	73.17
InfLLM	2048	100.00	94.00	85.20	95.20	99.80	96.40	93.00	88.45	94.08	22.10	83.80	74.60	57.20	83.37
UNIQUE	2048	100.00	99.80	99.20	99.60	100.00	99.80	99.00	99.15	99.96	82.28	91.07	75.60	57.40	92.53
Quest	512	99.40	94.60	91.00	97.20	95.80	53.00	96.55	96.65	99.00	53.24	87.60	74.00	55.80	84.14
H2O	512	99.80	98.00	2.60	97.60	84.60	0.00	44.45	14.65	66.88	20.94	68.33	74.40	55.80	56.00
InfLLM	512	92.60	86.00	76.00	88.60	98.40	43.80	85.60	79.40	87.52	10.12	74.40	74.20	56.20	73.30
UNIQUE	512	100.00	99.80	99.20	99.80	100.00	99.60	99.55	99.30	99.96	62.90	88.67	73.60	57.80	90.78

Table 2: Training-free RULER-32K results on Ministral-3-8B-Instruct-2512 (higher is better, $\uparrow$). **Budget** denotes the number of KV tokens each query is allowed to attend to at decoding time; full attention attends to all KV tokens.

KV budgets from 128 to 2048 pages. The two benchmarks expose complementary failure modes of prior methods. Quest (Tang et al., 2024), which estimates page importance from element-wise min/max of keys, remains competitive on RULER at all budgets (91.67 at 2048, 84.14 at 512) but degrades sharply on LongBench-Pro (21.72 at 512), indicating that min/max bounds are adequate for needle-style retrieval yet miss the broader reasoning context required by realistic long-context tasks. H2O (Zhang et al., 2023) shows the opposite pattern: its token-level eviction holds up on LongBench-Pro (29.04 at 512) but collapses on RULER (56.00 at 512, with 0 on NIAH_MULTIKEY_3 (MK3)), because permanently dropped KV cannot be recovered once a needle query arrives. InfLLM (Xiao et al., 2024) is the strongest baseline on LongBench-Pro (34.99 at 512) but trails Quest on RULER (73.30 vs. 84.14). **UNIQUE** is the only one that is uniformly strong

on both: at the default 512-page budget it reaches 36.58 on LongBench-Pro and 90.78 on RULER, recovering 97.0% and 97.5% of full attention while outperforming the best baseline by 1.59 and 6.64 points respectively. The advantage is consistent across languages and difficulty levels and persists at the tighter 128-page budget, where competing methods lose 40–90% of their full-attention score.

Sparsity-aware fine-tuning Table 3 reports LongBench-Pro English scores at the default 512-page budget after LoRA fine-tuning on LongAlign-10k from the same Ministral-3-8B-Instruct-2512 checkpoint. DSA (Liu et al., 2025) was originally designed for MLA; this paper adapts it to GQA by extending its lightning indexer from the MQA structure to a GQA structure, so that each KV head has its own indexer and selects pages independently within its group. All three sparse variants reach scores close to the fine-tuned dense

Method	Budget	English Split ($\uparrow$)
Full attention	–	36.55
InfLLM-v2	512	36.11
DSA	512	36.79
Proposed UNIQUE	512	37.25

Table 3: Sparsity-aware fine-tuning results on the English split of LongBench-Pro. All sparse methods and the full-attention baseline are LoRA fine-tuned on LongAlign-10k from the same Ministral-3-8B-Instruct-2512 checkpoint. **Budget** denotes the number of KV tokens each query is allowed to attend to at decoding time; full attention attends to all KV tokens.

Method	Brazilian		European		Avg
	EER	WER	EER	WER	
Full attention	18.95	20.97	13.22	19.85	18.25
Quest	22.97	23.02	19.37	22.30	21.92
InfLLM	56.02	48.19	69.53	57.12	57.72
Proposed UNIQUE	18.91	21.53	13.66	20.31	18.60

Table 4: Training-free long-form ASR results (lower is better, $\downarrow$) on Brazilian and European Portuguese long-form ($\sim$ 10-minute utterances) test sets at KV budget 512. Entity Error Rate (EER) requires every word of a reference entity to be transcribed verbatim.

baseline (36.55); notably, InfLLM-v2 (Zhao et al., 2025) jumps to 36.11 from its 33.82 training-free counterpart in Table 1, largely closing the gap left by training-free top- k . DSA (36.79) and UNIQUE (37.25) further surpass the dense baseline, suggesting that sparsity-aware training even removes some of the noise introduced by full-context attention. Among the three, UNIQUE attains the highest accuracy while keeping the cleanest training recipe: unlike DSA, UNIQUE requires neither an auxiliary distillation loss against the dense attention distribution nor materializing per-query attention scores from a separate indexer.

4.3 Long-Form Speech Recognition

Training-free results Long-form ASR is a particularly demanding stress test for sparse attention: a single dropped phrase or hallucinated word raises word error rate (WER), and the model cannot paraphrase its way around a missing page. Beyond standard WER, this paper also reports Entity Error Rate (EER, defined as $1 - \text{Entity Recall}$), which counts a reference entity as correctly recognized only when every constituent word is transcribed verbatim, providing a stricter measure of semantic integrity on information-dense long-form speech. Table 4 reports results on Brazilian and European Portuguese long-form ($\sim$ 10-minute) test sets at a 512-page KV budget. H2O fails to produce us-

Method	Brazilian		European		Avg
	EER	WER	EER	WER	
Full attention	18.95	20.97	13.22	19.85	18.25
InfLLM-v2	20.07	22.25	13.92	20.15	19.10
DSA	20.00	27.86	13.47	22.29	20.91
Proposed UNIQUE	18.54	20.75	13.09	19.17	17.89

Table 5: Sparsity-aware fine-tuning ASR results ($\downarrow$) on the same test sets as Table 4 at KV budget 512.

able transcriptions under this setting and is omitted from the table. InfLLM degrades the full-attention baseline by 40–55 absolute points on both metrics, suggesting that its selection heuristics, tuned for text, may need adaptation for other modalities. Quest is the strongest baseline (21.92 in average), still 3.67 points behind full attention. UNIQUE is the only method that matches dense attention on speech: its macro-average (18.60) is within 0.35 points of the dense baseline (18.25), and on Brazilian EER it even slightly improves over dense (18.91 vs. 18.95). Together with the text-domain results in Tables 1 and 2, this confirms that the mean-plus-std page score is modality-agnostic and remains reliable on inputs whose statistics differ substantially from natural text.

Sparsity-aware fine-tuning The speech LLM is itself fine-tuned from a text LLM, so sparsity-aware adaptation uses the same ASR data and recipe as the dense run; the full-attention numbers therefore match Table 4 and the comparison among sparse methods is fair. Both InfLLM-v2 and DSA perform well after fine-tuning, but UNIQUE reaches 17.89 macro-average, beating every baseline on every split and even slightly surpassing dense (18.25), without the local-window heuristic of InfLLM-v2 whose modality-specific prior may not always transfer. DSA closes most of the EER gap, showing that distilling the indexer recovers salient entity information, yet its WER remains noticeably higher; long-form ASR appears to expose the limits of attention distillation when even non-salient acoustic context must be retained for accurate transcription.

4.4 Decoding Efficiency

Attention kernel speedup Figure 2 reports the self-attention latency on a H100 with 32 query heads, 8 KV heads and head dimension 128, at page size $S=8$, KV budget 512, and batch size 80. UNIQUE reaches a $11.4\times$ speedup over dense FlashInfer at 32K context, and the gap widens with context length: the fixed-budget sparse pipeline grows sublinearly while dense scales linearly.

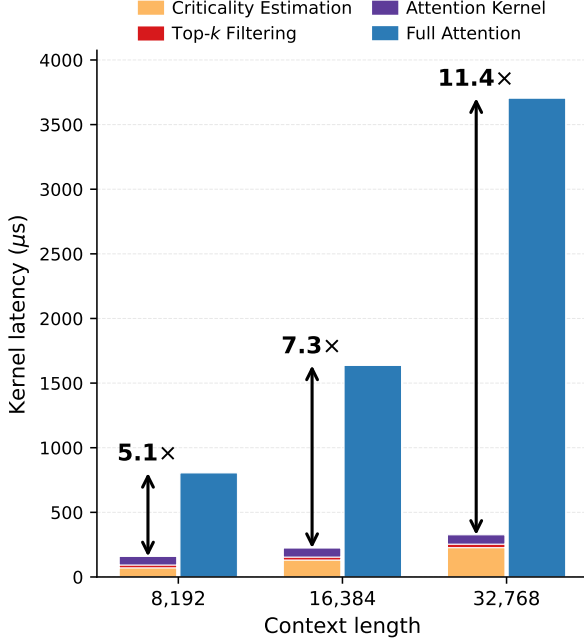


Figure 2: UNIQUE self-attention latency, which is broken down into criticality estimation, top- k filtering, and attention kernel; arrows indicate UNIQUE speedup over dense FlashInfer.

Scoring	Brazilian		European		Avg
	EER	WER	EER	WER	
UNIQUE w/o std	18.11	22.66	14.52	21.14	19.11
UNIQUE	18.91	21.53	13.66	20.31	18.60

Table 6: Training-free ablation of the std offset on the same ASR test sets as Table 4 (% , ↓).

End-to-end speedup. Figure 3 reports the end-to-end per-token decoding latency of UNIQUE on a 16-layer LLM with 32 query heads, 8 KV heads and head dimension 64, at batch size 80 with the default 512-token KV budget; both UNIQUE and the dense baseline run on vLLM (Kwon et al., 2023). At 32K context UNIQUE reaches up to $5.3\times$ end-to-end speedup over dense attention. The sparse pipeline keeps the per-token latency nearly constant because attention occupies only a small share of decoding in this model, masking the criticality estimator’s linear growth in context.

4.5 Ablations

Effect of the std offset Table 6 compares the training-free UNIQUE scoring (Eq. 3) against its mean-only ablation $q \cdot \text{mean}_p$ on long-form ASR with a 512-token KV budget. Removing the std term raises the macro-average from 18.60 to 19.11, with the largest regression on Brazilian WER (21.53 $\rightarrow$ 22.66), confirming that the std offset captures useful within-page variance that pure mean scoring under-weights.

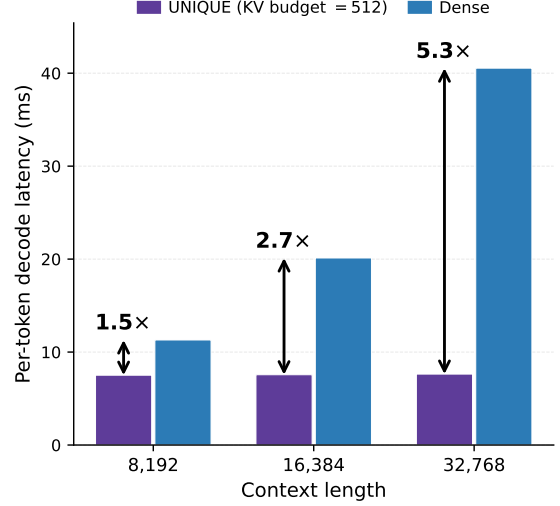


Figure 3: End-to-end per-token decoding latency under vLLM, batch 80, KV budget 512. Arrows mark UNIQUE speedup over the dense baseline.

Mask	Brazilian		European		Avg
	EER	WER	EER	WER	
UNIQUE	18.54	20.75	13.09	19.17	17.89
w/ hard mask	18.89	20.84	13.83	19.73	18.32

Table 7: Sparsity-aware fine-tuning ablation of the soft mask on the same ASR test sets as Table 4 (% , ↓).

Effect of the soft mask Table 7 ablates the sigmoid soft mask in sparsity-aware fine-tuning against a hard top- k mask under otherwise identical settings. The soft mask reduces the macro-average from 18.32 to 17.89 and improves every individual split. The gain comes from differentiability: the soft mask propagates gradients into the criticality estimator so the page-selection process itself adapts, whereas the hard mask only lets the model adapt to the already-selected pages.

5 Conclusions

This paper proposes UNIQUE, a universal top- k sparse attention that supports both training-free inference and sparsity-aware fine-tuning. UNIQUE scores KV pages by a simple yet effective mean-plus-std formula, and adapts the model via a soft-mask training scheme without auxiliary losses or model changes. On LongBench-Pro, RULER-32K and long-form ASR, UNIQUE is the only training-free method that matches dense attention across retrieval-style and realistic long-context tasks in text and speech; sparsity-aware fine-tuning further closes the residual gap. On a single H100, UNIQUE reaches $11.4\times$ attention-kernel and $5.3\times$ end-to-end speedup at 32K context, validating a single, modality-agnostic sparse attention design.

Limitations

KV-cache reduction is a broad and fast-moving area. This paper compares UNIQUE against representative training-free baselines (Quest, InfLLM, H2O) and trainable baselines (InfLLM-v2, DSA), but many other directions, including KV quantization and low-rank compression, are orthogonal and not compared here. A more exhaustive empirical sweep is left to future work.

This paper evaluates UNIQUE on text and speech LLMs, two of the modalities most affected by long-context inference. Extending the study to vision and vision-language models, which lie outside our expertise, is left to future work.

This paper considers UNIQUE under the fine-tuning regime, which is the setting most directly relevant to deployment and which allows comparison with prior fine-tunable baselines such as InfLLM-v2 and DSA. Methods that are designed for pre-training from scratch, such as Native Sparse Attention, are therefore not included in the comparison: they cannot be fairly evaluated under fine-tuning alone, and a from-scratch LLM pre-training is beyond our available compute. Although the design of UNIQUE itself does not preclude such a pre-training setting, that scenario is left to future work.

Ethical Considerations

This paper proposes UNIQUE to lower the inference cost of long-context LLMs in both text and speech. Today such inference remains expensive and is therefore most accessible to well-resourced groups; methods that reduce the per-token cost, such as UNIQUE, can broaden access to long-context capabilities and reduce the energy footprint of large-scale deployment. UNIQUE does not introduce additional sources of bias beyond those already present in the pre-trained LLM checkpoints and training data on which it is applied. All datasets, pre-trained models and software libraries used in this work are accessed under their stated terms for academic research, and the evaluation data does not contain personally identifying information or offensive content. AI assistants were used only to polish the writing of this paper.

References

Abdelrahman Abouelenin, Atabak Ashfaq, Adam Atkinson, Hany Awadalla, Nguyen Bach, Jianmin Bao, Alon Benham, Martin Cai, Vishrav Chaudhary, Con-

gcong Chen, and 1 others. 2025. Phi-4-mini technical report: Compact yet powerful multimodal language models via mixture-of-LoRAs. *arXiv preprint arXiv:2503.01743*.

Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, and 1 others. 2023. [GPT-4 technical report](#). *arXiv preprint arXiv:2303.08774*.

Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. 2023. GQA: training generalized multi-query transformer models from multi-head checkpoints. In *Proc. EMNLP*.

Yushi Bai, Xin Lv, Jiajie Zhang, Yuze He, Ji Qi, Lei Hou, Jie Tang, Yuxiao Dong, and Juanzi Li. 2024. LongAlign: A recipe for long context alignment of large language models. In *Proc. EMNLP (Findings)*.

Ziyang Chen, Xing Wu, Junlong Jia, Chaochen Gao, Qi Fu, Debing Zhang, and Songlin Hu. 2026. Long-Bench Pro: A more realistic and comprehensive bilingual long-context evaluation benchmark. *arXiv preprint arXiv:2601.02872*.

Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Naveen Sachdeva, Inderjit Dhillon, Marcel Blistein, Ori Ram, Dan Zhang, Evan Rosen, and 1 others. 2025. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261*.

Tri Dao. 2024. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *Proc. ICLR*.

Keqi Deng and Phil Woodland. 2025. Multi-head temporal latent attention. In *Proc. NeurIPS*.

Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, and 1 others. 2024. [The Llama 3 herd of models](#). *arXiv preprint arXiv:2407.21783*.

Yizhao Gao, Zhichen Zeng, DaYou Du, Shijie Cao, Peiyuan Zhou, Jiaxing Qi, Junjie Lai, Hayden So, Ting Cao, Fan Yang, and 1 others. 2025. SeerAttention: Self-distilled attention gating for efficient long-context prefilling. *Proc. NeurIPS*.

Cheng-Ping Hsieh, Simeng Sun, Samuel Krman, Shantanu Acharya, Dima Rekish, Fei Jia, Yang Zhang, and Boris Ginsburg. 2024. RULER: What’s the real context size of your long-context language models? *arXiv preprint arXiv:2404.06654*.

Huiqiang Jiang, Yucheng Li, Chengruidong Zhang, Qianhui Wu, Xufang Luo, Surin Ahn, Zhenhua Han, Amir H Abdi, Dongsheng Li, Chin-Yew Lin, and 1 others. 2024. Minference 1.0: Accelerating prefilling for long-context llms via dynamic sparse attention. In *Proc. NeurIPS*.

- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proc. SOSP*.
- Xunhao Lai, Jianqiao Lu, Yao Luo, Yiyuan Ma, and Xun Zhou. 2025. FlexPrefill: A context-aware sparse attention mechanism for efficient long-sequence inference. In *Proc. ICLR*.
- Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. 2024. SnapKV: LLM knows what you are looking for before generation. In *Proc. NeurIPS*.
- Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Daya Guo, and 1 others. 2024. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*.
- Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, and 1 others. 2025. Deepseek-v3. 2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*.
- Alexander H Liu, Kartik Khandelwal, Sandeep Subramanian, Victor Jouault, Abhinav Rastogi, Adrien Sadé, Alan Jeffares, Albert Jiang, Alexandre Cahill, Alexandre Gavaudan, and 1 others. 2026. Ministral 3. *arXiv preprint arXiv:2601.08584*.
- Se Jin Park, Julian Salazar, Aren Jansen, Keisuke Kinoshita, Yong Man Ro, and R. J. Skerry-Ryan. 2025. Long-form speech generation with spoken language models. In *Proc. ICML*.
- Zhiliang Peng, Jianwei Yu, Wenhui Wang, Yaoyao Chang, Yutao Sun, Li Dong, Yi Zhu, Weijiang Xu, Hangbo Bao, Zehua Wang, and 1 others. 2025. VibeVoice technical report. *arXiv preprint arXiv:2508.19205*.
- Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. 2023. Efficiently scaling transformer inference. In *Proc. ML-Sys*.
- Noam Shazeer. 2019. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*.
- Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. 2024. Quest: query-aware sparsity for efficient long-context LLM inference. In *Proc. ICML*.
- Chaojun Xiao, Pengl Zhang, Xu Han, Guangxuan Xiao, Yankai Lin, Zhengyan Zhang, Zhiyuan Liu, and Maosong Sun. 2024. InfLLM: Training-free long-context extrapolation for llms with an efficient context memory. In *Proc. NeurIPS*.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, and 1 others. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.
- Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. 2025. FlashInfer: Efficient and customizable attention engine for llm inference serving. *arXiv preprint arXiv:2501.01005*.
- Jiayi Yuan, Cameron Shinn, Kai Xu, Jingze Cui, George Klimiashvili, Guangxuan Xiao, Perkz Zheng, Bo Li, Yuxin Zhou, Zhouhai Ye, and 1 others. 2025a. BLASST: Dynamic blocked attention sparsity via softmax thresholding. *arXiv preprint arXiv:2512.12087*.
- Jingyang Yuan, Huazuo Gao, Damai Dai, Junyu Luo, Liang Zhao, Zhengyan Zhang, Zhenda Xie, Yuxing Wei, Lean Wang, Zhiping Xiao, Yuqing Wang, Chong Ruan, Ming Zhang, Wenfeng Liang, and Wangding Zeng. 2025b. Native sparse attention: Hardware-aligned and natively trainable sparse attention. In *Proc. ACL (1)*.
- Jintao Zhang, Chendong Xiang, Haofeng Huang, Jia Wei, Haocheng Xi, Jun Zhu, and Jianfei Chen. 2025a. SparseAttention: Accurate and training-free sparse attention accelerating any model inference. In *Proc. ICML*.
- Peitian Zhang, Zheng Liu, Shitao Xiao, Ninglu Shao, Qiwei Ye, and Zhicheng Dou. 2025b. Long context compression with activation beacon. In *Proc. ICLR*.
- Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark W. Barrett, Zhangyang Wang, and Beidi Chen. 2023. H2O: heavy-hitter oracle for efficient generative inference of large language models. In *Proc. NeurIPS*.
- Weilin Zhao, Zihan Zhou, Zhou Su, Chaojun Xiao, Yuxuan Li, Yanghao Li, Yudi Zhang, Weilin Zhao, Zhen Li, Yuxiang Huang, and 1 others. 2025. InfLLM-V2: Dense-sparse switchable attention for seamless short-to-long adaptation. *arXiv preprint arXiv:2509.24663*.

A Fused Criticality Estimation Kernel Benchmark

This appendix reports a microbenchmark of the fused criticality estimation kernel of UNIQUE. The kernel implements Eq. 6: a batched matrix multiplication for the per-head raw scores, followed by an on-the-fly addition of the per-head outer-product offset $\lambda \|q_g\|_2 \text{std}_p$ and a max reduction across the G query heads in a group. As a baseline, the same

KV len N	Pages P	Naive Implementation	Fused kernel	Speedup
8K	1,024	0.0670 ms	0.0388 ms	$1.73\times$
16K	2,048	0.0994 ms	0.0589 ms	$1.69\times$
32K	4,096	0.1597 ms	0.0990 ms	$1.61\times$
64K	8,192	0.2706 ms	0.1804 ms	$1.50\times$
128K	16,384	0.4984 ms	0.3422 ms	$1.46\times$

Table 8: Per-step latency of the criticality estimation kernel on an NVIDIA H100 GPU. The fused kernel computes Eq. 6 in a single CUDA launch; the naive implementation runs the same computation as three separate kernels.

Pages P	<code>torch.topk</code>	<code>flashinfer.top_k</code>	Top- k of UNIQUE	vs FlashInfer	vs PyTorch
1,024	0.031 ms	0.026 ms	0.013 ms	$2.0\times$	$2.4\times$
2,048	0.038 ms	0.026 ms	0.013 ms	$2.0\times$	$2.9\times$
4,096	0.068 ms	0.025 ms	0.014 ms	$1.8\times$	$4.8\times$
8,192	0.076 ms	0.027 ms	0.018 ms	$1.5\times$	$4.3\times$
16,384	0.102 ms	0.032 ms	0.022 ms	$1.4\times$	$4.6\times$

Table 9: Per-step latency of the top- k selection on an NVIDIA H100 GPU, with $k = 64$.

computation is performed naively as a batched matmul, a broadcast add, and a max reduction launched as three separate operations, which materializes the full $G \times P$ intermediate score tensor in HBM.

All measurements are taken on a single NVIDIA H100 GPU in bf16. The configuration matches the main experiments, with $H = 32$ query heads, $H_{kv} = 8$ KV heads (so $G = H/H_{kv} = 4$), head dimension $D = 128$, page size $S = 8$, $\lambda = 0.5$, and a batch of 32 decoding queries. The KV length N is swept from 8K to 128K tokens, which gives a number of pages P ranging from 1,024 to 16,384. Each entry is the median over 100 timed iterations after 20 warm-up runs.

Table 8 reports the per-step latency of the criticality estimation step alone. The fused kernel is consistently faster than the naive baseline across all KV lengths, with up to $1.73\times$ speedup at $N = 8K$. As N grows, the speedup ratio decreases because the matrix multiplication itself becomes the dominant cost and leaves less room for the bias and reduction steps to amortize, but the absolute time saving keeps growing and reaches 0.156 ms at $N = 128K$.

B Radix-based Top- k Kernel Benchmark

This appendix reports a microbenchmark of the radix-based top- k kernel of UNIQUE. The kernel takes a batch of bf16 score vectors of length P and returns, for each vector, the indices of the k largest entries via the two-round 8-bit radix selection described in Section 3.2. As

baselines, we compare against the top- k routines in PyTorch (`torch.topk`) and FlashInfer (`flashinfer.top_k`).

All measurements are taken on a single NVIDIA H100 GPU in bf16, with a batch of 32 decoding queries and $H_{kv} = 8$ KV heads, so each call performs $32 \times 8 = 256$ independent top- k selections in parallel. The selection budget is fixed to $k = 64$, corresponding to a token budget of 512 under page size $S = 8$. The number of pages P is swept from 1,024 to 16,384, covering KV lengths from 8K to 128K tokens. Each entry is the median over 300 timed iterations after 50 warm-up runs.

Table 9 reports the per-step latency of the top- k selection alone. The radix-based kernel is consistently faster than both baselines across all P , with up to $2.0\times$ speedup over FlashInfer and up to $4.8\times$ speedup over `torch.topk`. The gap over `torch.topk` widens with P , reflecting the close-to-linear scaling of the radix kernel.

C Hyper-parameters

Across all experiments, UNIQUE uses a page size of $S=8$. For the DSA baseline implemented in this paper, the lightning indexer follows the same GQA layout as the base LLM but with the head dimension reduced to $1/8$ of the original (e.g. from 128 to 16), so that its criticality-estimation cost is comparable to that of UNIQUE.

For text sparsity-aware fine-tuning on LongAlign-10k, Ministral-3-8B-Instruct-2512 is

fine-tuned with LoRA (rank $r = 64$, $\alpha = 128$) on all attention and MLP linear layers. Training runs for 5000 steps with a single sequence of length 65 536 per device, optimized by AdamW ($\beta_1=0.9$, $\beta_2=0.95$, $\epsilon=10^{-7}$, weight decay 0.01). The learning rate is 5×10^{-6} with 100 warm-up steps followed by cosine decay, gradient clipping at 1.0, and bf16 mixed precision with gradient checkpointing.

For speech sparsity-aware fine-tuning, a Qwen3-8B based speech LLM is built by attaching a 24-block Conformer audio encoder (attention dim 1024, 16 heads, FFN 1536, kernel size 3, $8\times$ time reduction, T5 relative position bias) to the language model. Fine-tuning runs on long-form Portuguese ASR for 10,000 steps, with sequences packed into 16,384-token batches and a global batch of 32 via gradient accumulation; utterances longer than 30 minutes are filtered out due to GPU memory limit. LoRA (rank $r = 320$, $\alpha = 640$) is applied to all attention and MLP linear layers. The learning rate is 4×10^{-5} with 100 warm-up steps and linear decay, and AdamW uses the same hyperparameters as above. Training is run with gradient clipping at 1.0, DeepSpeed ZeRO-1, and bf16 mixed precision.